

ETX Signal-X

Daily Intelligence Digest

Saturday, March 21, 2026

10

ARTICLES

15

TECHNIQUES

SQL injection vulnerability in WHERE clause allowing retrieval of hidden data [ES] [PortSwigger]

Source: securitycipher

Date: 26-Jul-2025

URL: <https://h0lm3s.medium.com/sql-injection-vulnerability-in-where-clause-allowing-retrieval-of-hidden-data-es-portswigger-e87f2606e013>

T1 Boolean-Based SQL Injection via Comment Truncation in Category Filter

Payload

```
GET /products?category=Gifts' --
```

Attack Chain

- 1 Identify the `category` parameter in the product listing endpoint (e.g., `/products?category=Gifts`).
- 2 Inject a single quote followed by double dash (`'--`) to terminate the string and comment out the rest of the SQL query.
- 3 Submit the payload and observe the response: products from the "Gifts" category are returned, including those not intended to be visible (e.g., unreleased products).

Discovery

Initial probing with a single quote (`'`) in the `category` parameter caused a 500 error, indicating unsanitized input and SQL error leakage. Further testing with SQL comment syntax (`--`) revealed the ability to bypass the `released = 1` filter.

Bypass

The double dash (`--`) comments out the remainder of the SQL statement, effectively removing the `AND released = 1` condition and exposing hidden records.

Chain With

Combine with UNION-based injection if column count is known to exfiltrate additional data. Use for privilege escalation or lateral movement if other injectable parameters exist.

T2

Boolean-Based SQL Injection with OR Condition to Dump All Products

Payload

```
GET /products?category=Gifts'%20OR%201=1--
```

Attack Chain

- 1 Target the `category` parameter in the product listing endpoint.
- 2 Inject a payload that closes the string, adds an `OR 1=1` condition, and comments out the rest: `Gifts' OR 1=1--` (URL-encoded as `Gifts'%20OR%201=1--`).
- 3 Submit the payload and observe that all products are returned, regardless of category or release status.

Discovery

After confirming SQL injection via comment truncation, the researcher tested logical conditions (`OR 1=1`) to force the query to always return true, resulting in a full product dump.

Bypass

The `OR 1=1` condition ensures the WHERE clause always evaluates to true, bypassing both category and release filters.

Chain With

Use as a base for extracting additional columns via UNION SELECT. Potential for authentication bypass if similar injection exists in login or access control logic.

T3

Error-Based SQL Injection Detection via Single Quote

Payload

```
GET /products?category='
```

Attack Chain

- 1 Send a single quote (``) as the value for the `category` parameter.
- 2 Observe application behavior; a 500 Internal Server Error is returned, indicating SQL syntax error and lack of input sanitization.

Discovery

Classic SQLi probing with a single quote to break query structure and trigger error handling, confirming injection point.

Bypass

N/A (detection technique, not exploitation).

Chain With

Use as a reconnaissance step before deploying more complex payloads. Combine with time-based or out-of-band payloads for further exploitation.

The Ghost in the Machine: How I Found IDORs That Were Hiding in Plain Sight

Source: securitycipher

Date: 04-Nov-2025

URL: <https://medium.com/@iski/the-ghost-in-the-machine-how-i-found-idors-that-were-hiding-in-plain-sight-0c26bca85c2f>

 No actionable techniques found

How I Hijacked an Instagram Account and Discovered Open Redirection Vulnerabilities

Source: securitycipher

Date: 10-Mar-2025

URL: <https://medium.com/@josekuttykunnelthazhebinu/how-i-took-over-an-instagram-account-using-broken-link-hijacking-found-open-redirections-on-c03b0c62add6>

T1 Broken Link Hijacking via Unclaimed Social Media Profile

Payload

```
https://www.google.com/url?q=https%3A%2F%2Fwww.instagram.com%2Fabc.com%2F&sa=D&sntz=1&usg=AOvVaw0Q0JuWGYwDDxPAuhKotxQ
```

Attack Chain

- 1 Identify embedded social media links (e.g., Instagram) on the target's subdomain.
- 2 Inspect the link and note the target username (e.g., instagram.com/abc.com) is unclaimed or broken.
- 3 Register a new Instagram account with the username matching the broken link (abc.com).
- 4 Revisit the target site and click the Instagram logo; the site redirects users to the attacker's newly created account.

Discovery

Manual browsing of the target's subdomain footer revealed a broken Instagram link. Inspection of the URL showed it pointed to an unregistered profile.

Bypass

Leverages the absence of account ownership validation for outbound social links. No verification that the linked profile is controlled by the legitimate company.

Chain With

Phishing: Use the hijacked social profile to impersonate the company for social engineering. Brand abuse: Spread malicious content under the company's perceived identity. Further recon: Identify other broken or unclaimed links for additional takeovers.

T2

Open Redirect on Login Page (return_to Parameter)

Payload

```
https://xyz.abc.com/login?return_to=https%3A%2F%2Fevil.com
```

Attack Chain

- 1 Locate the login endpoint with a `return\_to` parameter.
- 2 Set the `return\_to` parameter to a fully qualified external URL (e.g., https://evil.com).
- 3 Register or log in with valid credentials.
- 4 Upon successful authentication, the application redirects the user to the attacker-controlled domain.

Discovery

Parameter fuzzing on the login endpoint revealed that the `return\_to` parameter accepted external URLs and performed a redirect after login.

Bypass

No validation or whitelisting of redirect destinations. Accepts any external URL.

Chain With

Combine with phishing campaigns to steal credentials (post-login redirect to malicious site). Leverage for OAuth or SSO token theft if tokens are passed via URL. Use as a pivot for further attacks (e.g., session fixation, open redirect to XSS payload).

T3

Open Redirect on Registration Page (next Parameter)

Payload

```
https://xyz.abc.com/register?next=https://evil.com
```

Attack Chain

- 1 Locate the registration endpoint with a `next` parameter.
- 2 Set the `next` parameter to a fully qualified external URL (e.g., https://evil.com).
- 3 Complete the registration process.
- 4 After registration, the application redirects the user to the attacker-controlled domain.

Discovery

Parameter fuzzing on the registration endpoint revealed that the `next` parameter accepted external URLs and performed a redirect after registration.

Bypass

No validation or whitelisting of redirect destinations. Accepts any external URL.

Chain With

Combine with phishing or social engineering (e.g., "register to claim your prize" and redirect to malicious site). Use for OAuth/SSO token theft if tokens are passed post-registration. Pivot to further attacks (e.g., open redirect to XSS payload, session theft).

Insecure Direct Object Reference (IDOR) in a Government Portal

Source: securitycipher

Date: 24-Apr-2025

URL: <https://akashxak.medium.com/insecure-direct-object-reference-idor-in-a-government-portal-973758a23473>

T1 Unauthenticated IDOR via user_id Parameter in getprofile API

Payload

```
POST /sso_api/app/api/api_results.php
Content-Type: application/x-www-form-urlencoded

case=getprofile&user_id=1&app_id=4
```

Attack Chain

- 1 Authenticate to the government portal with any valid user account (or potentially without authentication if endpoint is exposed).
- 2 Intercept the following request in Burp Suite:
- 3 Modify the `user\_id` parameter to another value (e.g., 1 for admin, or enumerate sequentially: 3001, 3002, etc.).
- 4 Send the modified request.
- 5 Observe the response containing the profile data of the targeted user, including administrative or staff-level details.

Discovery

Manual inspection of API requests in Burp Suite after logging in; noticed `user\_id` was client-controlled and not tied to session. Enumerated IDs and observed data leakage for arbitrary users.

Bypass

No authentication or authorization checks enforced on the `user\_id` parameter; direct modification yields unauthorized access to other users' data, including privileged accounts.

Chain With

Use as a precursor for privilege escalation by targeting admin/staff accounts and harvesting sensitive information. Combine with password reset or account takeover vectors if additional endpoints are similarly vulnerable. Data harvested can be used for phishing, social engineering, or lateral movement within the organization.

How a Public readme.txt Led to a Critical WordPress CVE

Source: securitycipher

Date: 17-Dec-2025

URL: <https://samael0x4.medium.com/how-a-public-readme-txt-led-to-a-critical-wordpress-cve-dc8776454011>

T1 Enumerating WordPress Plugin Versions via Public readme.txt

Payload

```
curl -s 'https://xxxxxxx/wp-content/plugins/litespeed-cache/readme.txt' | sed -n '1,40p'
```

Attack Chain

- 1 Identify the target as a WordPress installation (e.g., via /wp-login.php or /wp-content/ present).
- 2 Access the plugin's public readme.txt file directly (e.g., /wp-content/plugins/litespeed-cache/readme.txt).
- 3 Parse the file for the "Stable tag" or version string (e.g., "Stable tag: 5.7").
- 4 Cross-reference the exposed plugin version with public CVE databases and exploit disclosures.
- 5 Confirm if the exposed version is affected by any critical vulnerabilities (e.g., CVE-2024-28000 for LiteSpeed Cache 5.7).
- 6 Use version certainty to inform or automate further exploitation if a matching CVE is found.

Discovery

Routine recon on WordPress targets; checking for unauthenticated exposure of plugin metadata files (readme.txt) as a habit. Triggered by the question: "What plugins might be exposed without authentication?"

Bypass

N/A – Relies on default public accessibility of readme.txt files in WordPress plugin directories.

Chain With

Chain with automated vulnerability scanners or exploit scripts that require confirmed plugin versions. Use as a precursor to targeted exploitation of known CVEs (e.g., RCE, privilege escalation) in the identified plugin version. Combine with other information disclosure vectors (e.g., changelogs, debug logs) to build a complete attack surface map.

Password Policy Bypass: Missing Server-Side Validation

Source: securitycipher

Date: 08-Jan-2026

URL: <https://sushil1337.medium.com/password-policy-bypass-missing-server-side-validation-947e91c8390e>

T1 Password Policy Bypass via Client-Side Only Validation

Payload

```
<button type="submit" disabled="disabled">Save</button> type "submit" "disabled"
```

Attack Chain

- 1 Initiate a password reset on the target application's "Forgot Password" page.
- 2 Receive and open the password reset link sent via email.
- 3 Enter a weak password (e.g., `sushil`). Observe that the "Save" button is disabled due to client-side validation.
- 4 Open Developer Tools → Elements tab and locate the submit button in the HTML.
- 5 Remove the attribute `disabled="disabled"` from the button or replace it with `enabled="enabled"`.
- 6 Click the now-enabled "Save" button to submit the weak password to the server.
- 7 Observe that the server accepts the weak password and confirms the reset.
- 8 Log in using the newly set weak password to verify the bypass.

Discovery

The researcher noticed that password strength enforcement was only present in the UI (disabled button, strength meter) and suspected missing server-side validation. Manual inspection of the HTML and form controls in DevTools confirmed the lack of backend enforcement.

Bypass

By directly manipulating the DOM to remove or alter the `disabled` attribute on the submit button, the attacker bypasses all client-side password policy checks and submits arbitrary weak passwords to the backend, which fails to validate them.

Chain With

Combine with account takeover vectors (e.g., intercepted or reused password reset links) to set trivially guessable passwords for high-value accounts. Use automated scripts or Burp Suite to mass-reset passwords to weak values across users if the endpoint lacks additional protections. Exploit in conjunction with weak session invalidation or missing password change notifications for stealthy persistence.

The “TMI” Endpoint: Why Unauthenticated Health Checks Are a Hacker’s Best Friend (And Why I Got 0...

Source: securitycipher

Date: 28-Jan-2026

URL: <https://zer0figure.medium.com/the-tmi-endpoint-why-unauthenticated-health-checks-are-a-hackers-best-friend-and-why-i-got-0-2945d21ea08f>

T1 Unauthenticated Health/Info Endpoint Leaking Backend Stack and Internal Details

Payload

```
GET /api/submissions/converter/health HTTP/1.1
Host: api.redactedgov.gov
Accept: application/json
```

Attack Chain

- 1 Use directory brute-forcing (e.g., ffuf, Dirbuster) against the API base URL to enumerate endpoints.
- 2 Identify unauthenticated endpoints such as `/health`, `/status`, `/info`, `/actuator/info`, or `/healthz`.
- 3 Send a GET request to the discovered endpoint (e.g., `/api/submissions/converter/health`) without authentication headers.
- 4 Parse the JSON response for sensitive metadata such as `implementationVersion`, `javaVersion`, internal file names, deployment dates, and operational status.
- 5 Use leaked data for targeted exploit development or further attack surface mapping.

Discovery

Fuzzed API endpoints for infrastructure/health-check URLs that are often left unauthenticated. Noticed that while main business logic endpoints enforced JWT authentication, the health endpoint did not, returning a verbose JSON payload.

Bypass

No authentication required; endpoint was excluded from access control checks, likely for operational monitoring. Standard auth bypass via direct access to infrastructure endpoints.

Chain With

Use `javaVersion` to search for and target version-specific CVEs (e.g., Java deserialization, Log4Shell, Spring4Shell, etc.). `implementationVersion` and deployment date can reveal outdated or abandoned deployments for prioritizing exploit attempts. Internal file names (e.g., `testCpcPlusValidationFile.json`) can be used for further fuzzing, LFI, or upload/overwrite attacks if other endpoints accept file operations. Combine with other low-severity leaks to escalate to RCE or privilege escalation if a matching exploit is found.

T2

Internal File Name Enumeration via Health Endpoint

Payload

```
GET /api/submissions/converter/health HTTP/1.1
Host: api.redactedgov.gov
Accept: application/json
```

Attack Chain

- 1 Access unauthenticated health endpoint as above.
- 2 Extract internal file names or validation file references from the JSON response (e.g., `validationFile: "testCpcPlusValidationFile.json"`).
- 3 Use the leaked file name to:

Discovery

Manual inspection of verbose health endpoint JSON output revealed internal file names and naming patterns, suggesting further attack vectors for file-based vulnerabilities.

Bypass

No authentication required; information disclosure due to lack of output sanitization on health endpoint.

Chain With

Use as a seed for LFI, path traversal, or arbitrary file upload attacks. Combine with endpoint enumeration to locate upload/download functionality. Potential for privilege escalation if file can be overwritten or used in application logic.

How I Find Another Arbitrary File Upload Vulnerability and Bypass the Patch Twice

Source: securitycipher

Date: 19-May-2024

URL: <https://medium.com/@zpbrent/how-i-find-another-arbitrary-file-upload-vulnerability-and-bypass-the-patch-twice-193eaaf0b8dc>

 No actionable techniques found

Post Board Writeup — MobileHackingLab

Source: securitycipher

Date: 23-Jan-2026

URL: <https://mohammadibnibrahim.medium.com/post-board-writeup-mobilehackinglab-ce045b6db36c>

T1 Deep Link to WebView XSS to JavaScriptInterface Method Invocation

Payload

```
PGLtZyBzcmM9eCBvbmVycm9yPVdlYkFwcEludGVyZmFjZS5wb3N0Q293c2F5TWVzc2FnZSgidGVzdGluZzt3aG9hbWkiKT4
```

Attack Chain

- 1 Identify exported `MainActivity` with deep link: `postboard://postmessage`.
- 2 Craft a payload containing a malicious `- 3 URL-Safe Base64 encode the payload (Android Base64.URL_SAFE variant).
- 4 Trigger the deep link via ADB:
- 5 The app decodes the payload, renders it in a WebView with JavaScript enabled, and executes the JavaScript via the image's `onerror` handler.
- 6 The JavaScript calls the exposed `WebAppInterface.postMessage()` method, passing the attacker's string.

Discovery

Manual APK decompilation and code review, focusing on exported activities, deep link handling, and JavaScriptInterface exposure in WebView. The researcher traced the data flow from deep link to WebView to JavaScript-exposed methods.

Bypass

Bypasses intended markdown-to-HTML rendering by injecting an image tag with an `onerror` handler, exploiting lack of sanitization in the markdown parser and leveraging the JavaScriptInterface exposure.

Chain With

Can be chained with Android intent redirection or other exported component abuse for lateral movement. If the WebView has additional JavaScriptInterface methods, further privilege escalation or data exfiltration may be possible.

T2

Command Injection via JavaScriptInterface to Runtime.exec()

Payload

```
<img src=x onerror=WebAppInterface.postCowsayMessage('testing;whoami')>
```

Attack Chain

- 1 Inject the above payload into the markdown message (delivered via deep link as in Technique 1).
- 2 When rendered, the WebView executes the JavaScript, calling `WebAppInterface.postCowsayMessage()` with the argument `testing;whoami`.
- 3 The `postCowsayMessage()` method passes this string to `CowsayUtil.runCowsay()`.
- 4 `runCowsay()` constructs a shell command: `/bin/sh -c "/data/data/com.mobilehackinglab.postboard/files/cowsay.sh testing;whoami"` and executes it with `Runtime.getRuntime().exec()`.
- 5 The `;` character breaks out of the cowsay command, and `whoami` is executed with the app's privileges, resulting in RCE.

Discovery

Source code review of the `JavaScriptInterface` methods revealed that `postCowsayMessage()` passes unsanitized user input directly to a shell command via `Runtime.exec()`. The ability to invoke this method from injected JavaScript in the WebView was confirmed via static analysis.

Bypass

No input sanitization or escaping is performed on the argument to `runCowsay()`. The attacker leverages shell metacharacters (`;`, `&&`, etc.) to break out of the intended command and execute arbitrary shell commands.

Chain With

Full RCE on the device in the app context. Possible privilege escalation if the app runs with elevated permissions or can access sensitive files. Persistence or lateral movement by dropping additional payloads via shell access.

T3

Abuse of URL-Safe Base64 in Deep Link Path for Payload Delivery

Payload

```
echo -n '<img src=x onerror=WebAppInterface.postMessage('testing;whoami')>' | base64  
| tr '+/' '-_'
```

Attack Chain

- 1 Prepare the malicious payload as a string (e.g., the XSS/command injection vector).
- 2 Encode the payload using URL-Safe Base64 (Android's Base64.URL_SAFE: `` for `+`, `` for `/`).
- 3 Place the encoded string as the path in the deep link: `postboard://postmessage/<encoded\_payload>`.
- 4 Trigger the intent to the exported activity as in Technique 1.
- 5 The app decodes the payload and processes it, leading to exploitation.

Discovery

Analysis of the `handleIntent()` method revealed the use of `Base64.decode(..., 8)` (URL_SAFE flag). Testing with standard Base64 failed, prompting investigation into the correct encoding variant.

Bypass

By using URL-Safe Base64, the attacker ensures the payload is accepted and correctly decoded by the app, bypassing failures caused by standard Base64 encoding.

Chain With

Enables reliable delivery of any payload to the vulnerable WebView/JavaScriptInterface chain. Can be used to deliver more complex serialized or binary payloads if the app logic allows.

Uncovering a Stored XSS Vulnerability on Drugs.com

Source: securitycipher

Date: 23-Sep-2024

URL: https://medium.com/@regan_temudo/uncovering-a-stored-xss-vulnerability-on-drugs-com-1748ccbcecc16

T1 Stored XSS via Question Submission on Drugs.com

Payload

```
<iframe onload=alert(document.domain)><iframe onload=alert(document.domain)> iframe onload=alert(document.domain)
```

Attack Chain

- 1 Navigate to <https://www.drugs.com/ask/> (Ask a Question feature).
- 2 Paste the payload into the "question" input field.
- 3 Submit the form/question.
- 4 The payload is stored and, when rendered on the site, triggers JavaScript execution in the context of any user viewing the question (alert on document.domain).

Discovery

Manual fuzzing of the question submission form with HTML/JS event handler payloads. The researcher targeted user-generated content fields likely to be rendered to other users, specifically testing for lack of HTML sanitization or output encoding.

Bypass

The payload leverages the <iframe> tag with an onload event handler, which is often overlooked by basic XSS filters that only blacklist <script> or inline event handlers on common tags (e.g., , <svg>). The use of <iframe> and event handler chaining increases the chance of bypassing naive or incomplete sanitization routines.

Chain With

Can be chained with session fixation, CSRF, or cookie theft if the rendered context includes authenticated users (e.g., moderators, support staff). If the question is indexed or referenced in notification emails, may enable stored XSS in additional contexts (e.g., email clients with HTML rendering).